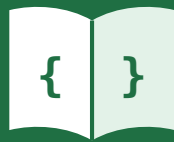

Module 02 — Configuration and Auto-Configuration

Lesson Notes

Spring Boot Edu



2026-09-23

Contents

1. Learning Goals	2
2. Concepts	2
2.1 Externalized Configuration	2
2.2 Relaxed Binding	2
2.3 Auto-Configuration	3
3. Step-by-Step Examples	3
3.1 Property Sources and Their Order	3
3.2 Typed Settings with @ConfigurationProperties	5
3.3 Injecting Single Values with @Value	6
3.4 Profiles and Profile Groups	7
3.5 spring.config.import	8
3.6 Writing Your Own Starter	9
3.7 Debugging with the Conditions Report	10
4. Common Mistakes and Best Practices	11
5. Summary	12
6. Further Reading	12

1. Learning Goals

By the end of this module you will be able to:

- Explain which source a setting comes from and which source wins
- Make settings typed, validated and immutable with `@ConfigurationProperties` records
- Use `@Value` where it fits and know its limits
- Use profiles, profile groups and profile-specific YAML documents
- Load extra configuration files with `spring.config.import`
- Understand how auto-configuration works and write your own starter
- Find out why an auto-configuration did or did not apply, using the conditions report

Prerequisites: Module 01 (IoC, `@Bean`, conditional beans) · **Estimated time:** 4 hours

2. Concepts

2.1 Externalized Configuration

The same application (the same JAR) runs with different settings on a developer machine, in tests and in production. Spring Boot keeps settings out of the code and reads them from many sources: YAML/properties files, environment variables, command-line arguments and more.

All sources sit in the Environment as an **ordered list**. When a key is requested, the first value found in the list wins. The important sources, **from lowest to highest** priority:

1. `application.yaml` (inside the JAR)
2. `application-{profile}.yaml` (inside the JAR)
3. `application.yaml` and `application-{profile}.yaml` outside the JAR
4. Operating-system environment variables
5. Java system properties (`-Dkey=value`)
6. Command-line arguments (`--key=value`)
7. In tests: `@SpringBootTest(properties = ...)`, `@TestPropertySource`

★ Tip

The rule is simple: **the more external and the more specific, the stronger**. Packaged defaults are the weakest, the command line is the strongest.

2.2 Relaxed Binding

Spring Boot matches key names flexibly. `support-email` in YAML, `supportEmail` in Java and `BOOKSTORE_STORE_SUPPORTEMAIL` as an environment variable are the same setting. The rule

for environment variables: dots become `_`, dashes are removed, letters are upper-cased.

2.3 Auto-Configuration

Spring Boot’s “magic” is that it looks at the libraries on the classpath and creates the beans you need for you. Every auto-configuration is really just a plain `@Configuration` class, guarded by **conditions**:

Condition	Meaning
<code>@ConditionalOnClass</code>	if the given class is on the classpath
<code>@ConditionalOnMissingBean</code>	if the application defined no bean of that type (“back-off”)
<code>@ConditionalOnProperty /</code> <code>@ConditionalOnBooleanProperty</code>	if a setting has a certain value

At startup Boot reads the `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration` files of all jars and evaluates the classes listed there.

i Note

Spring Boot 4 is modular. The database auto-configurations, for example, live in the `spring-boot-jdbc` module. If that module is not on the classpath, those auto-configurations are **not even candidates** (section 3.7).

3. Step-by-Step Examples

Because this module also builds our own starter, run it with `-am` (also-make):

```
./mvnw -pl modules/02-configuration/lesson -am spring-boot:run
```

The examples live in the `com.springbootedu.configuration` package. The starter is made of two separate Maven modules under `modules/02-configuration/starter/`.

3.1 Property Sources and Their Order

Goal: see where a value comes from, and learn the order of the sources by trying it.

The packaged defaults are in `application.yml`:

```

bookstore:
  store:
    name: Kitapçı
    support-email: destek@kitapci.example           # kebab-case in YAML → supportEmail
    ↪ in Java
    categories:
      - roman
      - bilim
      - yazılım
    shipping:
      free-from: "500.00"                         # quoted: unquoted 500.00 is a YAML
    ↪ float → 500.0
      delivery-time: 2d

```

PropertyOrigins walks the sources of the Environment in priority order and returns the name of the first one that contains the key:

```

@Component
public class PropertyOrigins {

    private final ConfigurableEnvironment environment;

    public PropertyOrigins(ConfigurableEnvironment environment) {
        this.environment = environment;
    }

    public String sourceOf(String key) {
        for (PropertySource<?> source : environment.getPropertySources()) { // highest
            ↪ priority first
            if (source.getName().equals("configurationProperties")) {
                continue; // Boot's combined view over all other
                ↪ sources – skip it
            }
            if (source.containsProperty(key)) {
                return source.getName();
            }
        }
        return "(not set)";
    }
}

```

Run it: override the same setting from the command line:

```

./mvnw -pl modules/02-configuration/lesson -am spring-boot:run \
  -Dspring-boot.run.arguments="--bookstore.store.name=Komut"

```

Expected output:

```
== 3.1 Property sources
bookstore.store.name = Komut ← commandLineArgs
bookstore.store.support-email ← Config resource 'class path resource [application.yaml]'
↪ ...
```

To try an environment variable: `BOOKSTORE_STORE_NAME="Ortam" ./mvnw -pl modules/02-configuration/lesson -am spring-boot:run`

Its test: `sources/PropertySourcesTest` checks that the command line overrides YAML and that the environment-variable name maps through relaxed binding.

3.2 Typed Settings with @ConfigurationProperties

Goal: collect related settings in one immutable record, convert them, and validate them at startup.

```
@ConfigurationProperties("bookstore.store")
@Validated // validate at startup – fail fast
public record StoreProperties(
    @NotBlank String name,
    @NotBlank @Email String supportEmail,
    @DefaultValue("TRY") Currency currency, // "TRY" → java.util.Currency,
    ↪ converted by Boot
    @DefaultValue List<String> categories, // empty list instead of null
    @Valid @NotNull Shipping shipping, // @Valid: validate the nested
    ↪ object as well
    @Nullable String banner) {

    /**
     * @param freeFrom orders from this amount ship for free
     * @param deliveryTime promised delivery time, e.g. 2d or 36h
     */
    public record Shipping(
        @NotNull @DecimalMin("0") BigDecimal freeFrom,
        @DefaultValue("3d") Duration deliveryTime) { // "2d", "36h", "PT12H" all work
    }
}
```

Boot converts text into the target types by itself: "2d" → Duration, "TRY" → Currency, a YAML list → List<String>. Thanks to @Validated, an invalid setting **stops the application from starting**. The error does not wait for a request in production.

Expected output:

```
== 3.2 @ConfigurationProperties
categories [roman, bilim, yazılım], currency TRY, free shipping from 500.00, delivery
↪ PT48H
```

Its test: `store/StorePropertiesTest` checks that a blank name, a negative fee and an invalid e-mail stop the application.

! Warning

In YAML, an unquoted `500.00` is a **floating-point number** and is read as `500.0`. Write sensitive values such as money in quotes: `free-from: "500.00"`.

★ Tip

`spring-boot-configuration-processor` generates metadata for your `@ConfigurationProperties` classes. The IDE then auto-completes these settings in `application.yaml` and shows their descriptions. It is enabled in every module of the course.

3.3 Injecting Single Values with @Value

Goal: see where `@Value` is useful and why it should be used sparingly.

```
@Component
public class StoreInfo {

    private final String name;
    private final String phone;
    private final int maxBooksPerOrder;

    public StoreInfo(@Value("${bookstore.store.name}") String name,
        ↪ // required
        @Value("${bookstore.store.phone:+90 212 000 00 00}") String phone,
        ↪ // with default
        @Value("#{2 + 3}") int maxBooksPerOrder) {
        ↪ // SpEL expression
        this.name = name;
        this.phone = phone;
        this.maxBooksPerOrder = maxBooksPerOrder;
    }
}
```

`${key:default}` gives a default value, `#{...}` is a SpEL expression. A missing key without a default stops the application from starting.

Expected output:

```
== 3.3 @Value
Kitapçı · +90 212 000 00 00 · max 5 kitap / books
```

Its test: legacy/StoreInfoTest

	@ConfigurationProperties	@Value
Several related settings	Yes, in one object	Each one separately
Validation	With @Validated	None
Relaxed binding	Full	Limited
IDE auto-completion	Yes (metadata)	No
SpEL	No	Yes

3.4 Profiles and Profile Groups

Goal: change settings per environment, and switch several profiles on with one name.

application-dev.yaml is loaded only while the dev profile is active and overrides the defaults:

```
# Lesson 3.4 — loaded only with the "dev" profile; overrides application.yaml
bookstore:
  store:
    name: Kitapçı (DEV)
    shipping:
      delivery-time: 1h
```

The same file can also contain a document, separated by ---, that applies to one profile only:

```
# Lesson 3.4 — a second YAML document, applied only when the "demo" profile is active
spring:
  config:
    activate:
      on-profile: demo
bookstore:
  store:
    banner: "DEMO — veriler gerçek değil / demo data"
```

Thanks to spring.profiles.group.local: dev, demo, the local profile switches on both dev and demo.

Run it:

```
./mvnw -pl modules/02-configuration/lesson -am spring-boot:run \
  -Dspring-boot.run.arguments="--spring.profiles.active=local"
```

Expected output:

```
== 3.4 Profiles
active profiles [local, dev, demo], banner: DEMO – veriler gerçek değil / demo data
```

Its test: profiles/ProfileGroupTest

3.5 spring.config.import

Goal: split settings into several files, and optionally load a machine-specific file that never goes into git.

application.yaml imports two files. The optional: prefix means “do not fail if the file is missing”:

```
config:
  import:
    - optional:classpath:campaigns.yaml
    - optional:file:./bookstore-local.yaml      # your machine-only overrides
    ↪ (git-ignored)
```

The imported campaigns.yaml:

```
# Lesson 3.5 – imported by application.yaml via spring.config.import
bookstore:
  campaigns:
    active:
      - code: OKULA-DONUS
        percent: 15
      - code: KITAP-FUARI
        percent: 25
```

Expected output:

```
== 3.5 spring.config.import
OKULA-DONUS → %15
KITAP-FUARI → %25
```

Its test: imports/ConfigImportTest

★ Tip

Create `modules/02-configuration/lesson/bookstore-local.yaml`, put for example `bookstore.store.name: My Bookstore` into it and run the application again. The file is in `.gitignore`. It is ideal for personal settings such as passwords.

3.6 Writing Your Own Starter

Goal: write an auto-configuration and turn it into a starter that is used through a single dependency.

A starter has two parts:

Module	Content
bookstore-greeting-autoconfigure	The code, the settings and the conditions
bookstore-greeting-spring-boot-starter	No code. It only brings the dependencies together.

First, the starter's settings:

```
@ConfigurationProperties("bookstore.greeting")
public record GreetingProperties(
    @DefaultValue("true") boolean enabled,
    @DefaultValue("Merhaba") String prefix,
    @DefaultValue("!") String suffix) {
}
```

Then the auto-configuration, guarded by conditions:

```
@AutoConfiguration
@ConditionalOnClass(GreetingService.class) // only if the class is
↳ on the classpath
@ConditionalOnBooleanProperty(name = "bookstore.greeting.enabled", matchIfMissing = true)
@EnableConfigurationProperties(GreetingProperties.class)
public class GreetingAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean // back off if the
    ↳ application has its own
    GreetingService greetingService(GreetingProperties properties) {
        return name -> properties.prefix() + ", " + name + properties.suffix();
    }
}
```

Finally, the class name goes into the imports file so that Boot can find it:

```
# Lesson 3.6 – Spring Boot loads every auto-configuration class listed here (one per
↪ line).
com.springbootedu.greeting.autoconfigure.GreetingAutoConfiguration
```

The lesson application only adds the starter dependency and sets `bookstore.greeting.prefix`: `Hoş geldiniz`. The bean just appears.

Expected output:

```
== 3.6 Our own starter
Hoş geldiniz, Ayşe!
```

Its tests: `starter/.../GreetingAutoConfigurationTest` tries every condition on its own (defaults, settings, back-off, switch-off, missing class, imports file). The lesson side has `starter/GreetingStarterUsageTest`.

! Important

Naming rule: official starters are called `spring-boot-starter-*`, third-party starters `*-spring-boot-starter`. Do not start the names of your own modules with `spring-boot`.

3.7 Debugging with the Conditions Report

Goal: find out why an auto-configuration did or did not apply.

When you start the application with `--debug`, Boot prints the **conditions evaluation report** to the console. The same information is available from code:

```
@Component
public class ConditionsExplainer {

    private final ConfigurableListableBeanFactory beanFactory;

    public ConditionsExplainer(ConfigurableListableBeanFactory beanFactory) {
        this.beanFactory = beanFactory;
    }

    public String explain(String autoConfigurationSimpleName) {
        Map<String, ConditionAndOutcomes> outcomes =
        ↪ ConditionEvaluationReport.get(beanFactory).getConditionAndOutcomesBySource();

        return outcomes.entrySet().stream()
```

```
        .filter(entry -> entry.getKey().endsWith(".") +  
            ↪ autoConfigurationSimpleName))  
        .findFirst()  
        .map(entry -> (entry.getValue().isFullMatch() ? "MATCHED: " : "SKIPPED: ")  
            + entry.getValue().stream()  
                .map(outcome -> outcome.getOutcome().getMessage())  
                .collect(Collectors.joining("; ")))  
        .orElse("NOT A CANDIDATE: its module is not on the classpath");  
    }  
}
```

Expected output:

```
== 3.7 Conditions report  
GreetingAutoConfiguration      MATCHED: @ConditionalOnClass found required class  
    ↪ '...GreetingService'; @ConditionalOnBooleanProperty  
    ↪ (bookstore.greeting.enabled=true) matched  
MessageSourceAutoConfiguration SKIPPED: ResourceBundle did not find bundle with basename  
    ↪ messages  
DataSourceAutoConfiguration    NOT A CANDIDATE: its module is not on the classpath
```

To see the full report:

```
./mvnw -pl modules/02-configuration/lesson -am spring-boot:run  
    ↪ -Dspring-boot.run.arguments="--debug"
```

Its test: diagnostics/ConditionsExplainerTest

i Note

The class is deliberately not called `AutoConfigurationReport`. Boot registers its own `ConditionEvaluationReport` bean under exactly that name, and reusing it causes a bean clash.

4. Common Mistakes and Best Practices

!! Caution

Do not put secrets such as passwords or API keys into `application.yaml` and push them to git. Use an environment variable, a local file imported with `optional:file:`, or a secret manager.

- **Do:** collect related settings in a `@ConfigurationProperties` record and validate them at startup with `@Validated`.

- **Don't:** spread the same key over dozens of `@Values` in different classes.
- **Do:** bind durations as `Duration` and sizes as `DataSize` (2d, 10MB).
- **Don't:** write decimal money values unquoted in YAML.
- **Do:** back off with `@ConditionalOnMissingBean` in your auto-configurations, and test every condition with `ApplicationContextRunner`.
- **Don't:** let component scanning pick up auto-configuration classes. They must be loaded only through the imports file.
- **Do:** when asking “why is this bean missing?”, look at the conditions report with `--debug` first.

5. Summary

- The Environment is an ordered list of sources. The more external and more specific source wins.
- `@ConfigurationProperties` records give typed, validated, IDE-friendly settings. `@Value` is for single values.
- Profile files, profile groups and on-profile documents change settings per environment.
- `spring.config.import` splits settings into several files, and `optional:` tolerates missing ones.
- An auto-configuration is a `@Configuration` class guarded by conditions. A starter only collects dependencies.
- The conditions report explains auto-configuration decisions.

Next step: [Exercises](#)

6. Further Reading

- [Spring Boot — Externalized Configuration](#)
- [Spring Boot — Profiles](#)
- [Spring Boot — Auto-configuration](#)
- [Spring Boot — Creating Your Own Auto-configuration](#)
- [Spring Boot — Configuration Metadata](#)